

Plexus Atlas — CompuCell3D

The second repository, and the first test of whether any of this transfers

1 August 2026

The twin of `atlas_jax_morph/atlas_note.pdf`. That one asked whether Plexus can say what one framework says. This one asks whether the *procedure* is a procedure.

1. Why a second repository, and why this one

One repository cannot show saturation. The measurement `plexus2.tex` App. E.1 asks for — *does decomposing a broad collection of frameworks converge on a compact reusable vocabulary?* — is a claim about what happens *across* frameworks. With $n = 1$ the curve is a shape, not an argument.

Target: CompuCell3D (Cellular Potts). Nominated by the `jax-morph` note before any of this work, on the grounds that it is *the biggest step away from everything we have*: a cell is not a point with per-cell state, it is a **set of lattice sites**. Its position is a derived centroid. Its dynamics are Metropolis flips of site ownership. There is no per-cell equation to subtract.

That is the point. Every measurement in the `jax-morph` atlas rested on cells being points and a differential test being a subtraction. **If the procedure is general it survives this; if it is secretly a particle-framework procedure, this is where it breaks.**

The prediction — made before running anything, and the reason this note exists — is in `TRANSFER.md`.

2. Phase 0 — what transferred, measured rather than asserted

`STATUS.md` in the `jax-morph` atlas claimed ~60% of the Python was instrument and would copy as-is. Forking it is the test of that claim, and it came out slightly better than claimed.

file	edits	why
<code>record.py</code>	none	schema, ladder and the twelve rules are about the record, not the target. Self-test passes unchanged.
<code>saturation.py</code>	none	the ledger scores against the frozen baseline; nothing in it knows what a cell is. Self-test passes.
<code>atlas.py</code>	none	blast radius, rung, commit-or-revert — all HERE-relative.
<code>registry_view.py</code>	none	reads <code>plexus.operators</code> . Produced the same 52-contract baseline from the new folder.
<code>agents/</code>	none	the six roles describe a job, not a repository.
<code>run_spec.py</code>	5 lines	output paths only (<code>log/atlas_cc3d/</code> , <code>graphs_data/atlas_cc3d/</code>).
<code>oracle.py</code> , <code>inventory.py</code> , <code>paper.py</code>	rewrite	target-specific by construction: the reference, its mechanism scan, its PDF.

Five of the seven instruments needed zero edits. That is the strongest evidence so far that the apparatus is a method rather than a one-off, and it cost about ten minutes to establish.

3. The oracle — where the risk actually is

TRANSFER.md predicted the oracle would be the largest risk and a *target* risk rather than a *procedure* risk. It is, and here is the state of it.

gate	state	
installs	done	CompuCell3D 4.10.0 from its own conda channel into an isolated environment (/workspace/.conda_envs/cc3d-oracle), py312.
imports	done	after supplying libGL (conda-forge libgl-devel); the container is headless and CompuCell3D links its graphics stack unconditionally.
pure-Python model definition	done	cc3d.core.PyCoreSpecs — a simulation can be built without XML, which is what makes a scriptable oracle possible at all.
seed exposed	done	PottsCore(random_seed=...), so determinism is at least <i>expressible</i> .
headless run	BLOCKED	see below.
deterministic at a fixed seed	not started	cannot be tested until it runs.

The blocker is a bug in CompuCell3D 4.10.0, not in the plan. Driving it from PyCoreSpecs through service_cc3d reaches persistent_globals.get_custom_settings_path(), which does Path(custom_settings_source) where custom_settings_source is the *list of specs* rather than a simulation file path:

```
TypeError: argument should be a str or an os.PathLike object ... not 'list'
```

Three ways out, in the order they will be tried: an earlier release (4.9.0 / 4.8.0) where the pure-Python path may predate the regression; the .cc3d project-file route via cc3d_runScript.sh, which is the officially supported headless entry point and does not touch that code path; or setting the custom-settings path explicitly before the call.

Nothing downstream can be trusted until this gate passes, and that is the whole discipline: jax-morph was chosen *because* it ran, and the note said so. A target that cannot be run reduces the atlas to the Okuda situation — wrong operator, wrong parameter or wrong reading, with no way to tell which.

4. What is not started

Phases 1–7. The mechanism inventory, the specs, the differential tests and the ledger all wait on the oracle gate. **The one thing worth doing before it passes** is the instrument gap TRANSFER.md identifies: the ledger’s implementation class detects a contract claimed twice *within* one run, not *across* runs — so a CompuCell3D mechanism matching a jax-morph *candidate* would read as **new** when it is really a second sighting. That is a defect that only becomes visible at the second repository, which is exactly the kind of thing this exercise exists to surface.

Where things are. `TRANSFER.md` — the prediction, written first. `../atlas_jax_morph/STATUS.md` — what the first atlas produced and what was reusable. `_state/registry_baseline.json` — the same frozen 52 contracts, so the two measurements are comparable.